第 8 章 幻觉、遵守与验证

> 本章目标：认清 AI 会以哪三种方式失败，掌握"说得出 → 跑得通 → 测得过 → 现场看得见"四级验证，养成"宁可报告未验证，不美化结果"的习惯。

> 要点：AI 的输出是"看起来最像正确答案的文字"，不是"核对过的事实"。它的自信程度和正确程度之间没有稳定关系——说得越流畅，越值得你检查。

8.1 三类失败

AI 出错的方式不止一种，混在一起讲就很难对症下药。分成三类：

一、幻觉（hallucination）

它编造了不存在的东西，而且编得很像真的。常见形态：

- **编事实**：某本书里"有一个著名论断"、某个研究"表明"——翻遍原文找不到。
- **编引用**：给你作者、年份、页码齐全的参考文献，论文根本不存在。
- **编 API**：推荐一个 `client.getLatestModel()`，你们这个库的文档里压根没有。
- **编路径**：告诉你配置在 `config/app.yaml`，你的项目里没有这个文件。

幻觉最麻烦的地方在于：**它的形式是正确的**——格式对、语气对、术语对，只是内容是空的。

二、知识过时

模型的训练有截止时间，此后发生的事它不知道。它不知道时会怎么办？要么说"我不了解"，要么——更常见——**用旧知识自信地补上**，于是你得到一个"上个版本的世界"。

编程里这一点尤其致命：库升级了、函数改名了、参数顺序变了，模型给的旧写法一跑就报错。所以**以你本地实际安装的版本和官方文档为准**，永远比"AI 说应该这样写"更可靠。

三、不遵守（instruction following 失败）

它没有编造，也没有过时，就是**没按你说的做**：

- **漏要求**：你列了 5 条验收，它做了 4 条，还很自然地汇报"已完成"。
- **擅自扩展**：你只让它改一个函数的注释，它顺手"优化"了三个文件的命名。
- **破坏约束**：你说了"不许引入新依赖"，它加的 import 引用了你没装的包。
- **改掉不该动的东西**：为了通过测试而修改测试，问题看起来消失了。

现象	原因	对策
编造事实、引用、API、文件路径	生成目标是"连贯、像真的"，而不是"真实"	逐条核查；让它给出来源并**打开**来源；关键 API 查官方文档
用旧版本写法，报错	训练数据有截止时间	以本地文档和实际运行结果为准；把版本号写进需求
漏做要求、擅自扩大范围	长上下文里多条要求会衰减；它倾向"多做一点"	要求逐条对照验收清单；要求 diff；限制改动范围
测试通不过就改测试	目标函数被优化成"让检查消失"	明确"不许修改测试"；失败样本要保留
嘴上说"已验证"	它没有运行环境，或运行了但没看结果	要求贴命令和输出；自己复跑

8.2 为什么会这样

不必理解神经网络的全部细节，理解三点就够了：

第一，它是按"下一个最可能的词"生成的。 训练目标是让输出读起来连贯、合理。当"真实"和"连贯"冲突时，模型往往选连贯——训练数据里，流畅的错误答案比"我不知道"更常见。

第二，上下文里没有的信息，它只能猜。 你问"我们项目的数据库配置写在哪"，如果没把文件内容给它，它不会说"你没告诉我"，而是按"通常这样的项目会放在哪"来回答。**它给的是行业惯例，不是你的项目。**

第三，训练数据与工具结果冲突时，它可能选错。 模型"记得"某个函数是这个签名，而你实际运行报错说不是——它有可能反过来解释你的报错。**运行时的事实永远优先于模型的记忆。**

类比：一个读过很多书、但从没进过你实验室的人给你指路。他描述得很流利、很有条理，但他说的是"据说那栋楼长这样"，不是"我刚走过去看了"。

8.3 验证分级（本章重点）

这是整个课程里最应该带进工作习惯的一条。把"做完了"拆成四级，逐级往上，每一级都需要更强的证据。

第 1 级：说得出

> AI："脚本已经写好，逻辑是按照你说的实现的。"

这是**零证据**。它只说明模型生成了这段文字。你不能据此向任何人汇报"已完成"。

第 2 级：跑得通

> 你："请贴出运行命令和完整输出。"

```
$ python ik_demo.py --target 200 100 150
IK solved in 12 iterations, residual = 0.0003
```

这比第一级强得多——至少有一次真实的执行。但它只说明**这一组输入**能跑通。换个目标点呢？超出工作空间呢？关节限位呢？

第 3 级：测得过

写几个固定的用例：正常点、边界点、非法点（比如目标点在机械臂够不到的地方，期望是**明确报错而不是乱动**），跑一遍全绿。

但这里有个关键认识：

> 要点：**测试通过 ≠ 需求正确。** 测试是照着你写的标准去检查的。如果标准本身写错了，测试会忠实地通过一个错误的结果。

比如要求"机械臂移动到目标点"，测试只检查"函数返回成功"，而没检查"机械臂真的到了"——测试全绿也没用。

第 4 级：现场看得见

在真实的教室、网络和设备上跑一遍。这一级会发现前三级全部漏掉的问题：手机屏幕上的按钮被挡住、教室 Wi-Fi 不稳定导致超时、舵机在真实负载下抖动更大、投屏时字体太小。

对任何"要给别人看"的东西，第 4 级是唯一的验收标准。前三级都是**通往它的手段**，不是终点。

报进度时请标明等级。 "脚本写好了"和"本地跑过一次，没做边界测试"是完全不同的两句话。后者诚实，前者含糊，而含糊会被听成"已经好了"。

8.4 事实核查

一手来源优先

来源类型	可信度	用法
官方文档、源码、标准文本	高	优先查，以它为准
官方仓库的 issue / 变更日志	较高	查"这个行为是 bug 还是设计"
技术 blog、教程	中	可参考思路，结论要回到一手来源核对
论坛回答、二手转载	低	只当线索
AI 的回答	视情况	**当线索，不当结论**

关键动作：它给出来源之后，你要真的打开那个来源。 链接可能 404、可能指向无关页面、也可能页面存在但内容被曲解。不打开就等于没核查。

还有一个容易被忽视的点：

> **模型自报版本不可靠。** 问它"你是什么版本"，回答来自提示词或训练数据，不是运行时信息。要确认版本，看官方发布说明、看 API 返回的元数据字段、看你自己安装的包版本。

8.5 代码验证

代码的验证有固定套路，逐条做：**读 diff**（不接整文件重写，看它到底改了哪几行）、**跑测试、复现 bug**（修复前先能稳定复现，修复后再确认消失；不能复现的修复等于没修复）、**试边界输入**（空值、零、负数、超长串、够不到的目标点，AI 写的代码通常只覆盖正常路径）、**看日志**（没崩不代表没出错，警告可能被吞掉了）。

评测（eval）基础

当你要判断"改完之后到底变好了没有"，需要一组固定的评测：

- **固定一组输入与期望输出**，比如 20 道题，每道有标准答案或可判断的对错。
- **改完再跑同一组**，换题就失去可比性。
- **小模型和大模型用同一套评测**，否则你比较的不是模型，是两套题。
- **记录失败样本，不删失败的测试**。失败的用例最有价值，删掉它就失去了"下次别再犯"的依据。

一个 20 条的小评测，比"我感觉好像变好了"有用得多。

8.6 数据与隐私

- 不要把 **API key、密码、身份证号、学生真实姓名与联系方式、未公开的商业代码** 贴进你不信任的服务。
- 需要 AI 帮忙排查时，**先脱敏**：把真实数据换成结构相同的假数据（张三 → 学生A，手机号 → 138****0000）。如果数据结构本身就能反推出人，那就不要上传。
- 本地模型的边界：它能保证"数据不出机器"，但能力更弱、更容易出错，验证要求不能因此降低——反而要更严。

8.7 教师立场：宁可报告未验证

这一条不是技术，是纪律，但它是本课程最重要的一条。

课程纪律：如果你没有验证，就明确说"未验证"。不要用"应该可以""理论上没问题""已经完成了"来填补验证的空缺。

理由很实际：机器人会真的动，一个"应该没问题"的逆解可能让机械臂撞到桌子；你说"做好了"，别人会基于这句话安排下一步，含糊的汇报把风险转移给了其他人；而一旦被发现美化过结果，之后你说的每句话都会被怀疑——**可信度是累积的，也会一次性清零。**

诚实汇报的模板：

> **已验证**：主路径在本地跑通（附命令与输出），测试 3/5 通过。 **未验证**：边界输入未测；教室无线网络下未测。 **已知问题**：目标点超出工作空间时没有报错，会静默返回上一次的解。 **下一步**：补边界用例。

这样的汇报没人会觉得你做得不好，只会觉得你可靠。

课后题

1. 找一个真实 AI 回答中的幻觉：可以是教材里的例子、你自己和 AI 的对话，或老师提供的案例。写出**原文、错误之处、你如何发现的**，并归类为"编事实 / 编引用 / 编 API / 编路径"中的哪一种。
2. 用 8.3 节的四级验证，给你的小组项目写一份当前状态表：哪些功能在第 1 级，哪些到了第 2 级，有没有到第 3、4 级的?
3. 说明为什么"测试通过"不等于"需求正确"，并举一个测试全绿但功能错误的例子（可以自己构造）。
4. AI 告诉你"这个库的 `move_to()` 支持第三个参数用于设置速度"。请写出你会怎样核查这条信息，至少列出三步。
5. 下面是一句汇报："主要功能都做完了。"请把它改写成符合 8.7 节模板的版本（已验证 / 未验证 / 已知问题 / 下一步）。
6. 开放题：如果你发现小组里有人为了"看起来完成了"而删掉了一个失败的测试，你会怎么处理？谈谈理由，不必只考虑"批评"这一种方式。